I want you to generate a python code for a part of the project. The project is a student helper that ensures students can move around campus safely at night. The idea is that a student requests help through phone and provides : current location, destination and time of departure(when they want to leave current location). and then a reciever/volunteer student can accept the request to help. I want you to generate a function for this program that handles the safety request, validates it and then publishes the request to volunteers. the format of request is assumed to be:
1. location -> str
2. destination -> str
3. time -> datetime

Show less

Here's a clean, structured Python function that:

- Validates the safety request
- Ens checks for missing/invalid data
- Publishes the request (simulated with a placeholder you can replace, e.g., MQTT / API / DB)

✅ Function: Handle & Publish Safety Request

Python Run

```python
from datetime import datetime, timedelta

class InvalidRequestError(Exception):
    """Custom exception for invalid safety requests."""
    pass


def handle_safety_request(location: str, destination: str, departure_time: datetime):
    """
    Validates and publishes a student safety request.
```

```
    Args:
        location (str): Current location of the student
        destination (str): Destination of the student
        departure_time (datetime): Planned departure time

    Returns:
        dict: Published request data

    Raises:
        InvalidRequestError: If validation fails
    """

    # -------------------------
    # 1. VALIDATION
    # -------------------------

    # Validate location
    if not isinstance(location, str) or not location.strip():
        raise InvalidRequestError("Invalid location. Must be a non-empty string.")

    # Validate destination
    if not isinstance(destination, str) or not destination.strip():
        raise InvalidRequestError("Invalid destination. Must be a non-empty string.")

    # Validate time type
    if not isinstance(departure_time, datetime):
        raise InvalidRequestError("Invalid time. Must be a datetime object.")

    now = datetime.now()

    # Check if time is in the past
    if departure_time < now:
        raise InvalidRequestError("Departure time cannot be in the past.")

    # Optional: limit how far in future (e.g., max 24h)
    if departure_time > now + timedelta(hours=24):
        raise InvalidRequestError("Departure time is too far in the future.")

    # Prevent same location and destination
    if location.strip().lower() == destination.strip().lower():
        raise InvalidRequestError("Location and destination cannot be the same.")

    # -------------------------
    # 2. CREATE REQUEST OBJECT
    # -------------------------

    request = {
        "location": location.strip(),
        "destination": destination.strip(),
        "departure_time": departure_time.isoformat(),
        "status": "pending",
        "created_at": now.isoformat()
```

```
    }

    # -------------------------
    # 3. PUBLISH REQUEST
    # -------------------------

    publish_to_volunteers(request)

    return request


# -------------------------
# PUBLISHING FUNCTION (MOCK)
# -------------------------

def publish_to_volunteers(request: dict):
    """
    Simulates sending the request to volunteers.
    Replace this with MQTT / WebSocket / API call.
    """

    # Example: print (replace with real system)
    print("📢 New Safety Request Published:")
    print(request)

    # Example alternatives:
    # mqtt_client.publish("safety/requests", json.dumps(request))
    # send_to_api(request)
    # websocket.broadcast(request)
```

💡 How It Works (Quick Study Version)

Step 1 — Validate

- Strings must not be empty
- Time must be:
  - A `datetime`
  - Not in the past
  - Not too far in the future

Step 2 — Create Request

- Clean input (`strip()`)
- Add metadata (`status` , `created_at`)

Step 3 — Publish

- Sends request to volunteers (currently mocked)

🧠 Memory Hook

Think of this function like a **security guard at the gate**:

- ❌ Invalid request → rejected immediately
- ✅ Valid request → stamped & sent out to helpers

🔧 If you want next step

I can help you extend this with:

- Matching volunteers based on distance
- Accept/decline request logic
- Real-time updates (MQTT / sockets)
- Database storage

Just tell me 👍

I want a simpler version that is easier to track

Got it — let's strip it down to the **core logic only**, so it's easy to follow and debug.

✅ Simple Version

Python Run

```python
from datetime import datetime

def handle_safety_request(location, destination, departure_time):
    # --- 1. VALIDATE INPUT ---

    if not location or not isinstance(location, str):
        print("Invalid location")
        return None

    if not destination or not isinstance(destination, str):
        print("Invalid destination")
        return None

    if not isinstance(departure_time, datetime):
        print("Invalid time")
        return None
```

```
    if departure_time < datetime.now():
        print("Time is in the past")
        return None

    # --- 2. CREATE REQUEST ---

    request = {
        "location": location,
        "destination": destination,
        "time": departure_time
    }

    # --- 3. PUBLISH (SIMPLIFIED) ---

    print("New request sent to volunteers:")
    print(request)

    return request
```

💡 Why this is easier

- No custom exceptions
- No extra rules (like 24h limit)
- Uses simple `print` instead of external systems
- Straight **3-step flow**

🧠 Memory Hook

"Check → Pack → Send"

1. ✔️ Check inputs
2. 📦 Pack into dictionary
3. 📡 Send (print)

🔧 If you want slightly better (but still simple)

I can next show you:

- how to plug this into MQTT
- or how to store requests in a list/database
- or how volunteers accept requests

Just tell me 👍